

梯度法求解时间最优量子比特重置 (σ_x)

最优控制问题直接法求解工具箱

黄泓博

2026 年 9 月 7 日

摘要

现成封装的商用最优控制求解器（如基于 CasADi/IPOPT 的直接法）通常不会输出迭代的中间过程，这使得无法直观地调试收敛过程。本工作面向含 σ_x 驱动的量子比特重置这一时间最优控制问题，从零编写了一个直接法梯度求解器，它以输出每次迭代的状态、协态、控制与成本为设计目标，从而能够定位并解决收敛过程中的问题。工具箱将具体的最优控制问题与求解算法解耦：只需按抽象接口提供一个问题，即可用同一套算法求解。

本文档首先介绍梯度法的理论（庞特里亚金极值原理）、有约束情况的两种处理（惩罚函数与投影梯度法）、历史累计的方向优化（Adagrad、RMSprop、经典动量、Nesterov、Adam）以及步长搜索准则；随后给出要解决的量子比特重置控制论问题；最后介绍为解决梯度消失与初值敏感问题而引入的正则化与同伦延拓（数值延拓 / prolong），以及整个工具箱的程序结构和可替换问题的方法。

1 待求最优控制问题的抽象与工具箱结构

本工具箱的核心思想是：把"待求的最优控制问题"抽象成一个接口，把"求解算法"实现为通用的可复用组件。只要一个具体问题满足该接口，就可以用同一套梯度求解器去解它。

抽象接口 (`ocp.AbstractOCP`)：一个最优控制问题只需提供以下内容——

- 状态维数、控制维数、总时长 τ ；
- 时间格点 t_{grid} 与初态 x_0 ；
- 初始控制的猜测 $u^{(0)}(t)$ ；
- 目标泛函及其对控制的（离散）梯度 $(J, \frac{\partial J}{\partial u})$ ；
- 控制的可行域（长方体约束）及其投影。

求解器只依赖这些内容，完全不关心问题的物理细节。

具体问题: `ocp.QubitReset` (本工作求解的含 σ_x 量子比特重置问题) 与 `ocp.ControlExample` (一个独立的、可解析求解的示例问题，见第7节末尾)。二者实现同一接口，故同一求解器可直接替换使用。

求解算法 (`solver.GradientSolver`): 在下文各节中描述，集成了投影梯度、历史累计方向优化与多种步长搜索准则。

同伦延拓外壳 (`continuation.HomotopyContinuation`): 一个与具体问题无关的延拓层，用于处理梯度消失与初值敏感问题，见第8节。

因此，要解决另一个最优控制问题，只需实现 `ocp.AbstractOCP` 接口，不需要修改任何求解算法。

2 梯度法

¹ 在打靶法和配点法之前，先介绍梯度法，该方法既不离散化控制，也不离散化状态，而是直接通过梯度下降法迭代整个过程中的控制。

2.1 无约束的情况

将最优控制问题写成哈密顿形式

$$L_{\text{aug}} = L + \boldsymbol{\lambda}^T (\dot{\mathbf{x}} - \mathbf{f}) \quad (2.1)$$

$$H = \boldsymbol{\lambda}^T \dot{\mathbf{x}} - L_{\text{aug}} = \boldsymbol{\lambda}^T \mathbf{f} - L \quad (2.2)$$

于是欧拉-拉格朗日方程变为协态方程（哈密顿正则方程之二）

$$\dot{\boldsymbol{\lambda}} = -\frac{\partial H}{\partial \mathbf{x}} \quad (2.3)$$

在上式和运动方程（哈密顿正则方程之一）的约束下，求解哈密顿量 H 的极值

$$\frac{\partial H}{\partial \mathbf{u}} = 0 \quad (2.4)$$

进一步地，庞特里亚金最优化原理还告诉我们，此时哈密顿应为常数。

在写出最优控制问题的哈密顿量后，梯度法的主要步骤可以写作

1. 猜测初始控制 $\mathbf{u}^{(k=0)}(t)$.
2. 将 $\mathbf{u}^{(k)}(t)$ 带入哈密顿正则方程，积分得到 $\mathbf{x}^{(k)}(t)$ 和 $\boldsymbol{\lambda}^{(k)}(t)$.

¹ 本小节内容参考最优化与最优控制（第二版），赫孝良，葛照强，西安交通大学出版社，2015.

3. 用此时的 $\mathbf{u}^{(k)}(t)$, $\mathbf{x}^{(k)}(t)$ 和 $\lambda^{(k)}(t)$ 计算此时的梯度 $\frac{\partial H}{\partial \mathbf{u}}^{(k)}$.
4. 利用梯度迭代控制 $\mathbf{u}^{(k+1)}(t) = \mathbf{u}^{(k)}(t) + h^{(k)} \frac{\partial H}{\partial \mathbf{u}}^{(k)}$, 其中 $h^{(k)}$ 为步长.
5. 计算梯度的大小 $\left| \frac{\partial H}{\partial \mathbf{u}}^{(k)} \right|$, 判断其是否小于设定精度 ϵ . 若否, 则返回第二步.
6. 符合条件, 终止.

例 1: 控制论问题

- **运动方程:** $\dot{x} = -x^2 + u$.
- **边界约束:** $x(0) = 10$, $t = 1$ 末态时刻为自由边界.
- **目标函数:** $J = \int_0^1 \frac{1}{2}(x^2 + u^2) dt$.

解: 首先写出哈密顿量

$$L = \frac{1}{2}(x^2 + u^2) \quad (2.5)$$

$$L_{\text{aug}} = \frac{1}{2}(x^2 + u^2) + \lambda(\dot{x} + x^2 - u) \quad (2.6)$$

$$H = \lambda(-x^2 + u) - \frac{1}{2}(x^2 + u^2) \quad (2.7)$$

和协态方程

$$\dot{\lambda} = -\frac{\partial H}{\partial x} = 2x\lambda + x \quad (2.8)$$

$t = 1$ 时刻为自由边界, 给出自然边界条件 $\left. \frac{\partial L_{\text{aug}}}{\partial \dot{x}} \right|_{t=t_f} = 0$ 给出

$$\lambda(1) = 0 \quad (2.9)$$

求梯度得

$$\frac{\partial H}{\partial u} = \lambda - u \quad (2.10)$$

迭代求解. 事实上, 因为该问题较为简单, 首次迭代的效果就已经很好了. $\square$

该例 1 对应工具箱中的 `ocp.ControlExample` 问题. 它没有末态成本, 而是一条运行成本积分, 因此哈密顿量中的运行项 $(-L)$ 不可忽略. 用同一求解器可求解该问题, 以验证所谓"可替换问题"的抽象性.

梯度法可以拓展到高阶情况, 即考虑 H 的高阶导数. 除了提高阶数之外, 还有共轭梯度法、变尺度方法等推广自静态最优的方法. 此略.

梯度法的缺点也是明显的: 它仍然需要利用哈密顿量 H 及其梯度 $\frac{\partial H}{\partial \mathbf{u}}$ 的解析表达式; 另一方面, 其收敛效果受步长选取的影响较大, 在后文中会提供一些步长搜索准则来缓解这个缺点.

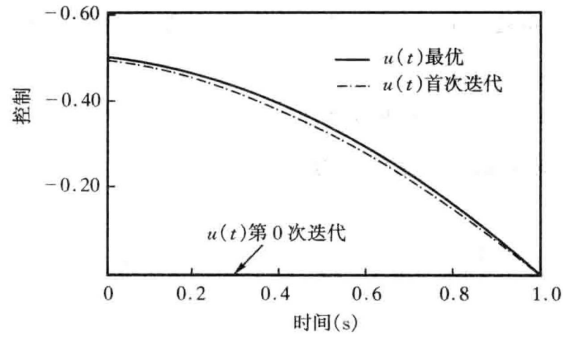


图 13.1

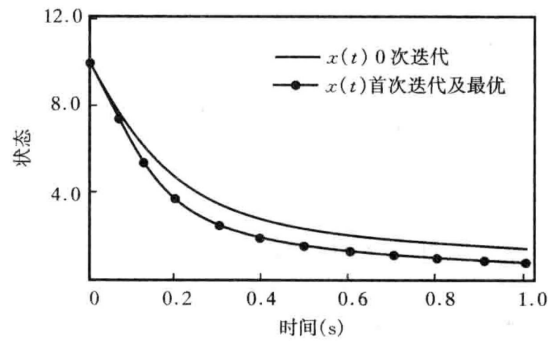


图 1: 例 1 首次迭代得到的结果

3 有约束的情况：惩罚函数

将边界约束和路径约束转化为突破约束的惩罚函数，对目标函数进行增广

$$J_{\text{aug}} = K + \int_{t_0}^{t_f} L dt + \phi^T G \phi + \int_{t_0}^{t_f} \sum_i g_i |C_i|^P \Theta(C_i) dt \quad (3.1)$$

其中 $\Theta(x)$ 为阶跃函数, G 为一正（负）定对称矩阵, g 为一全正（负）向量, 当它们趋近于 $\pm\infty$ 时, 求解增广目标函数最小（大）值的问题将等价于在边界约束和路径约束下求解原目标函数的最小（大）值. 在实际数值计算中, 调节 G 和 g 究竟取多大取决于我们的精度要求, 一般取 $P = 2$.

例 2: 控制论问题

- 运动方程: $\dot{x} = u$.
- 边界约束: $x(0) = 1, x(1) = 0$.

- 目标函数: $J = \int_0^1 u^2 dt$.

解: 将末态改为自由边界, 约束写进增广目标函数中

$$J_{\text{aug}} = Gx(1)^2 + \int_0^1 u^2 dt \quad (3.2)$$

哈密顿量及正则方程

$$H = u^2 + \lambda u \quad (3.3)$$

$$\dot{\lambda} = -\frac{\partial H}{\partial x} = 0 \quad (3.4)$$

$t = 1$ 时刻自由边界的自然边界条件 $\lambda(1) = 2Gx(1)$, 解得 $x(t) = 1 - \frac{G}{1+G}t$. 当 $G \rightarrow \infty$ 时, $x(1) \rightarrow 0$, 最优控制 $u(t) = -1$. $\square$

惩罚函数法在数值上往往**不稳定**: 当惩罚因子 g 较大时, 目标函数会变得刚性, 容易导致震荡甚至发散. 因此本工具箱默认采用下一节更稳定的**投影梯度法**来纳入路径约束.

4 投影梯度下降法

投影梯度下降法把约束的处理从"惩罚"改为"投影". 其迭代为

$$\mathbf{u}^{(k+1)}(t) = \mathbf{Proj}\left(\mathbf{u}^{(k)}(t) + h^{(k)} \frac{\partial H^{(k)}}{\partial \mathbf{u}}\right) \quad (4.1)$$

其中 $\mathbf{Proj}$ 为向控制的可行域 (对量子比特重置问题是一个长方体 $\lambda_x \in [0, \lambda_x^{(m)}]$, $\lambda_z \in [0, \lambda_z^{(m)}]$) 的投影. 数学上可以证明, 投影操作不影响收敛性; 对于可分离的长方体约束, 它甚至也不怎么额外消耗性能.

需要注意"定义在边界处的梯度": 当某个控制分量已经到达其上(下)界, 且梯度继续把它往外推时, 该分量该方向的"有效梯度"应为零. 这相当于把控制约束的互补松紧条件(KKT活动集)并入梯度. 本工具箱在投影梯度的基础上, 通过活动集检测(`kktActiveSet` 选项)实现这一点.

5 历史累计的方向优化

在上一节的梯度下降法中, 算法迭代的方向仅由梯度决定 $\mathbf{d}^{(k)} = \frac{\partial H}{\partial \mathbf{u}}^{(k)}$. 这可能会遇到一些困难, 如图2鞍点跨越就是其中的一个.

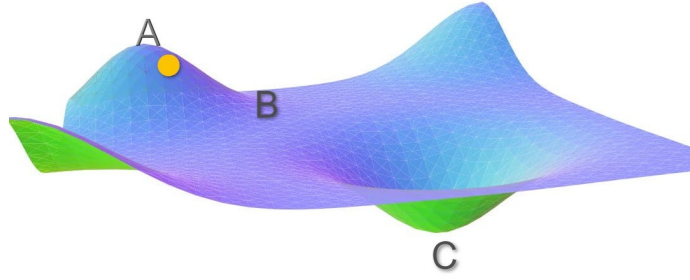


图 2: 鞍点跨越的示意图. 当程序从初值 A 点处开始优化, 最开始梯度指引的下降方向很明确, 但是当下降到 B 点后, 就进入了所谓的高原区 (Plateau Region), 需要进行鞍点跨越 (Saddle Point Traversal) 才能进一步下降.

累计梯度的历史来改进优化的方向会是一件有所帮助的事情. 下面介绍工具箱实现的各种方向优化算法. 对量子比特重置问题, 梯度法是收敛较慢的, 且倾向于在高原区停滞; 采用这些"历史累计的方向"能显著改善跨越鞍点和高原区的能力.

记号约定: 设 k 为迭代次数, $\mathbf{g}^{(k)} \stackrel{\text{def}}{=} \frac{\partial H}{\partial \mathbf{u}}^{(k)}$ 为当前步的梯度方向 (工具箱内部以"目标泛函的梯度" $\frac{\partial J}{\partial \mathbf{u}}$ 存储, 二者只差一个符号, 不影响方向), $\mathbf{d}^{(k)}$ 为实际采用的更新方向, $\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} + h^{(k)} \mathbf{d}^{(k)}$. 分量运算均为逐元素进行.

5.1 自适应梯度 (Adaptive Gradient, Adagrad)

Adagrad 维护历史梯度平方的累计

$$\mathbf{G}^{(k)} = \mathbf{G}^{(k-1)} + (\mathbf{g}^{(k)})^2 \quad (5.1)$$

并用其平方根对当前梯度缩放, 从而自动减小更新步长, 适应不同参数的稀疏性:

$$\mathbf{d}^{(k)} = -\frac{\mathbf{g}^{(k)}}{\sqrt{\mathbf{G}^{(k)}} + \epsilon} \quad (5.2)$$

其中 ϵ 为防止除零的小量. Adagrad 适合稀疏梯度问题, 但累计项会不断增大, 导致步长单调递减, 可能退化失效.

5.2 均方根传播 (Root Mean Square Propagation, RMSprop)

RMSprop 通过对历史平方梯度采用指数加权移动平均 (EMA)，限制累计项增长：

$$\mathbf{E}^{(k)} = \beta_2 \mathbf{E}^{(k-1)} + (1 - \beta_2) (\mathbf{g}^{(k)})^2 \quad (5.3)$$

$$\mathbf{d}^{(k)} = -\frac{\mathbf{g}^{(k)}}{\sqrt{\mathbf{E}^{(k)} + \epsilon}} \quad (5.4)$$

其中 β_2 为二阶矩衰减系数. 它保留了 Adagrad 的自适应能力，又避免了过度衰减.

5.3 经典动量 (Classical Momentum, CM)

Polyak 重球法引入动量：

$$\mathbf{d}^{(k)} = \beta_1 \mathbf{d}^{(k-1)} + (1 - \beta_1) \mathbf{g}^{(k)} \quad (5.5)$$

其中 β_1 为一阶矩 (动量) 衰减系数. 动量项帮助克服鞍点和高原区，减少震荡.

5.4 Nesterov 加速梯度 (Nesterov Accelerated Gradient, NAG)

NAG 在计算梯度前先用动量" 预看" 一步：

$$\mathbf{d}^{(k)} = \beta_1 \mathbf{d}^{(k-1)} + (1 - \beta_1) \left. \frac{\partial H}{\partial \mathbf{u}} \right|_{\mathbf{u}^{(k)} + \beta_1 \mathbf{d}^{(k-1)}} \quad (5.6)$$

预看点可能已越过梯度为零的平坦区，能提供更有用的方向信息 (例如指向鞍点的下坡方向)，在鞍点和高原区表现更优.

5.5 自适应矩估计 (Adaptive Moment Estimation, Adam)

Adam 同时维护梯度的一阶矩 (动量) 与二阶矩 (历史) 的 EMA，并进行偏差校正：

$$\mathbf{v}^{(k)} = \beta_1 \mathbf{v}^{(k-1)} + (1 - \beta_1) \mathbf{g}^{(k)} \quad (5.7)$$

$$\mathbf{m}^{(k)} = \beta_2 \mathbf{m}^{(k-1)} + (1 - \beta_2) (\mathbf{g}^{(k)})^2 \quad (5.8)$$

$$\hat{\mathbf{v}}^{(k)} = \frac{\mathbf{v}^{(k)}}{1 - \beta_1^k} \quad \hat{\mathbf{m}}^{(k)} = \frac{\mathbf{m}^{(k)}}{1 - \beta_2^k} \quad (5.9)$$

$$\mathbf{d}^{(k)} = -\frac{\hat{\mathbf{v}}^{(k)}}{\sqrt{\hat{\mathbf{m}}^{(k)} + \epsilon}} \quad (5.10)$$

Adam 兼具自适应性与动量的特点.

与线搜索配合的注意点：动量类方法的 $\mathbf{d}^{(k)}$ 并不一定是下降方向。工具箱在与线搜索配合时，会先判断当前方向是否下降（即 $(\frac{\partial J}{\partial \mathbf{u}})^T \mathbf{d}^{(k)} < 0$ ），若非下降方向则本轮不更新。这正是动量方法所需的额外程序设计（见 `GradientSolver` 源码）。

6 步长搜索准则

² 步长的选取对梯度法的收敛很重要。这不仅体现在变步长方法可以加快收敛的速度，不合适的步长选取甚至会导致迭代在最优解附近反复横跳、难以收敛的情况。一般只采用一些非精确搜索的准则即可。

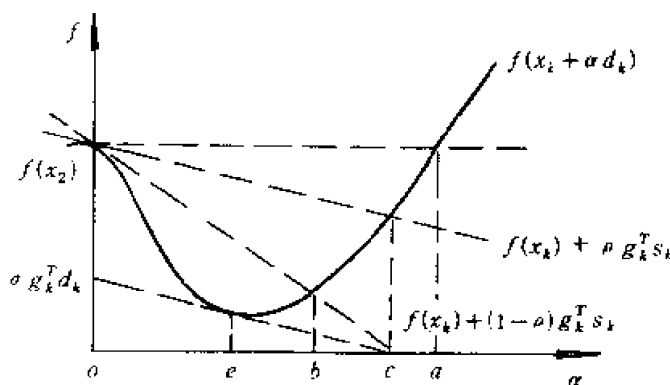


图 3: 准则的示意图. 区间 $(0, a)$ 为让函数值下降的区间, 即梯度下降法能正常工作的区间. 区间 $(0, c)$ 为简单准则的可接受区间. 区间 (b, c) 为 Armijo-Goldstein 准则的可接受区间, 极值点有可能被排除在其外. 区间 (e, c) 为 Wolfe-Powell 准则的可接受区间.

单纯形准则要求步长 $h^{(k)}$ 不应该太靠近区间的右边界 $a^{(k)}$:

$$f(x^{(k)} + s^{(k)}) \leq f(x^{(k)}) + \rho f'(x^{(k)})s^{(k)} \quad (6.1)$$

其中 $\rho \in (0, \frac{1}{2})$. 这被称为后退方法 (backtracking), 通常简单地把步长减半即可.

Armijo-Goldstein 准则在简单准则基础上, 为防止步长太小给出额外条件 (不靠近左边界):

$$f(x^{(k)} + s^{(k)}) \geq f(x^{(k)}) + (1 - \rho)f'(x^{(k)})s^{(k)} \quad (6.2)$$

²本小节内容参考最优化理论与方法, 袁亚湘, 孙文瑜, 科学出版社, 1997.

Wolfe-Powell 准则给出更简单的左边界条件:

$$f'(x^{(k)} + s^{(k)})d^{(k)} \geq \sigma f'(x^{(k)})d^{(k)} \quad (6.3)$$

其中 $\sigma \in (\rho, 1)$. 对于非精确搜索一般取 $\rho = 0.1, \sigma = 0.4$. 本工具箱的 `solver.LineSearch` 实现了这三种准则, 并支持二分法插值.

7 要解决的控制论问题

要解决的控制论问题为给定擦除时间, 找到擦除精度最高的擦除路径.

- **状态与控制:** 系统的状态 $\mathbf{p} = (p_e, p_r, p_i)$, 控制为 (λ_x, λ_z) .
- **运动方程:** $\dot{\mathbf{p}} = \mathbf{P}\mathbf{p} + \mathbf{p}_0$.
- **边界约束:** $\mathbf{p}(0) = (0.5, 0, 0)$; $t = \tau$ 末态时刻为自由边界, 有自然边界条件见下文.
- **路径约束:** 控制的路径约束 $\lambda_x \in [0, \lambda_x^{(m)}], \lambda_z \in [0, \lambda_z^{(m)}]$.
- **目标函数:** $J = |\mathbf{p}(\tau)|$.

其中

$$\mathbf{P} = \begin{pmatrix} -\gamma(4\Gamma_z \sin^2 \theta + (\Gamma_+ + \Gamma_-)(1 + \cos^2 \theta)) & \gamma(4\Gamma_z - \Gamma_+ - \Gamma_-) \cos \theta \sin \theta & -2\Lambda \sin \theta \\ \gamma(4\Gamma_z - \Gamma_+ - \Gamma_-) \cos \theta \sin \theta & -\gamma(4\Gamma_z \cos^2 \theta + (\Gamma_+ + \Gamma_-)(1 + \sin^2 \theta)) & 2\Lambda \cos \theta \\ 2\Lambda \sin \theta & -2\Lambda \cos \theta & -\gamma(4\Gamma_z + \Gamma_+ + \Gamma_-) \end{pmatrix} \quad (7.1)$$

$$\mathbf{p}_0 = \begin{pmatrix} \gamma(2\Gamma_z \sin^2 \theta + \frac{1}{2}(\Gamma_+(1 - \cos \theta)^2 + \Gamma_-(1 + \cos \theta)^2)) \\ -\gamma(2\Gamma_z \cos \theta \sin \theta - \frac{1}{2}(\Gamma_+ \sin \theta(\cos \theta - 2) + \Gamma_- \sin \theta(\cos \theta + 2))) \\ -\Lambda \sin \theta \end{pmatrix} \quad (7.2)$$

$$\Gamma_z(t) = \pi \sin^2 \theta(t) \quad (7.3)$$

$$\Gamma_{\pm}(t) = \cos^2 \theta(t) \frac{1}{2} J(2\Lambda(t)) (\coth(\Lambda(t)) \pm 1) \quad (7.4)$$

$$J(\omega) = \pi \omega \frac{\omega_c^2}{\omega_c^2 + \omega^2} \quad (7.5)$$

其中

$$\theta(t) = \arctan \frac{\lambda_x}{\lambda_z}, \quad \Lambda(t) = \sqrt{\lambda_x^2 + \lambda_z^2} \quad (7.6)$$

密度矩阵的正定性要求会给出状态的路径约束，主方程会保证其在物理上自然满足，故不特意写出。

哈密顿量

$$H = \boldsymbol{\mu}^T (\mathbf{P}\mathbf{p} + \mathbf{p}_0) - L \quad (7.7)$$

正则方程

$$\dot{\mathbf{p}} = \mathbf{P}\mathbf{p} + \mathbf{p}_0 \quad (7.8)$$

$$\dot{\boldsymbol{\mu}} = -\mathbf{P}^T \boldsymbol{\mu} \quad (7.9)$$

当 $L = 0$ （纯末态成本）时，自由边界的自然边界条件给出

$$\boldsymbol{\mu}(\tau) + \frac{\mathbf{p}(\tau)}{|\mathbf{p}(\tau)|} = 0 \quad \implies \quad \boldsymbol{\mu}(\tau) = -\frac{\mathbf{p}(\tau)}{|\mathbf{p}(\tau)|} \quad (7.10)$$

梯度

$$\frac{\partial H}{\partial(\lambda_x, \lambda_z)} = \boldsymbol{\mu}^T \left(\frac{\partial \mathbf{P}}{\partial(\lambda_x, \lambda_z)} \mathbf{p} + \frac{\partial \mathbf{p}_0}{\partial(\lambda_x, \lambda_z)} \right) - \frac{\partial L}{\partial(\lambda_x, \lambda_z)} \quad (7.11)$$

其中

$$\frac{\partial(\theta, \Lambda)}{\partial(\lambda_x, \lambda_z)} = \begin{pmatrix} \frac{\lambda_z}{\Lambda^2} & -\frac{\lambda_x}{\Lambda^2} \\ \frac{\lambda_x}{\Lambda} & \frac{\lambda_z}{\Lambda} \end{pmatrix} \quad (7.12)$$

梯度消失问题：由于系统的动力学是耗散的，而成本是末端函数 $|\mathbf{p}(\tau)|$ ，当擦除时间 τ 较长时，末端对初态（进而对初始控制）的敏感性会下降，导致梯度 $\frac{\partial H}{\partial \mathbf{u}}$ 在迭代初期趋于零，梯度法难以启动。这正是引入下一节正则化与同伦延拓的动机。

可替换性示例：`ocp.ControlExample`（本节例 1）实现了同一接口，但它是一个标量状态/控制、带运行成本的问题。同一个 `solver.GradientSolver` 无需任何修改即可求解它，这体现了"把待求问题抽象为可替换工具"的设计。

8 正则化与同伦延拓

为解决梯度消失与初值敏感问题，本工具箱提供**同伦延拓**（数值延拓 / prolongation）机制：把原问题嵌入到一族参数化的问题中，从一个容易求解的子问题出发，逐步把参数推进到原问题的参数，每一步以上一步的最优解作为初始猜测（warm start）。

在量子比特重置问题中，工具箱支持两种延拓族（由 `QubitReset.homotopyName` 选择）：

1. **时域延拓** (`homotopyName = 'tau'`): 从较短的擦除时间 τ_0 逐步增大到目标 τ 。短时间时末端对初态仍敏感, 梯度不消失, 先解出再外推, 从而绕过初始位置梯度消失的问题。
2. **成本权重延拓** (`homotopyName = 'kappa'`): 为成本添加一个正则化运行项, 并从 $\kappa = 0$ 逐步增大到 $\kappa = 1$:

$$J = \kappa |\mathbf{p}(\tau)| + (1 - \kappa) \frac{1}{\tau} \int_0^\tau \left(\frac{\lambda_x}{\lambda_x^{(m)}} + \frac{\lambda_z}{\lambda_z^{(m)}} \right) dt \quad (8.1)$$

$$K = \kappa |\mathbf{p}(\tau)| \quad (8.2)$$

$$L = L_r = (1 - \kappa) \frac{1}{\tau} \left(\frac{\lambda_x}{\lambda_x^{(m)}} + \frac{\lambda_z}{\lambda_z^{(m)}} \right) \quad (8.3)$$

$\kappa = 1$ 即回到原本的最优化问题。此时自然边界条件变为

$$\boldsymbol{\mu}(\tau) + \kappa \frac{\mathbf{p}(\tau)}{|\mathbf{p}(\tau)|} = 0 \quad (8.4)$$

哈密顿量 (含运行项) 与梯度分别为

$$H = \boldsymbol{\mu}^\top (\mathbf{P}\mathbf{p} + \mathbf{p}_0) - (1 - \kappa) \frac{1}{\tau} \left(\frac{\lambda_x}{\lambda_x^{(m)}} + \frac{\lambda_z}{\lambda_z^{(m)}} \right) \quad (8.5)$$

$$\frac{\partial H}{\partial(\lambda_x, \lambda_z)} = \boldsymbol{\mu}^\top \left(\frac{\partial \mathbf{P}}{\partial(\lambda_x, \lambda_z)} \mathbf{p} + \frac{\partial \mathbf{p}_0}{\partial(\lambda_x, \lambda_z)} \right) - (1 - \kappa) \frac{1}{\tau} \left(\frac{1}{\lambda_x^{(m)}}, \frac{1}{\lambda_z^{(m)}} \right) \quad (8.6)$$

当 κ 较小时, 正则化项主导, 成本函数对初态的控制梯度更强, 问题更"好解"; 随着 $\kappa \rightarrow 1$, 末端成本权重恢复, 逐步回到原问题。

抽象实现: `continuation.HomotopyContinuation` 是与具体问题无关的延拓外壳。它只做两件事: 调用问题对象的 `setHomotopy` 设置当前参数, 以及把上一步解作为下一步的初始猜测交给内部求解器。因此, 任何支持延续参数的问题都可直接套用, 无需修改延拓层本身。

9 程序结构

工具箱的目录结构 (MATLAB 包) 如下:

```

1  SigmaX_OCP/
2  src/
3      +ocp/                % 抽象问题接口与具体问题
4      AbstractOCP.m        % 抽象接口 (可替换问题的契约)
5      QubitReset.m         % 含 sigma_x 的量子比特重置问题
6      ControlExample.m     % 另一个可替换问题 (例1)

```

```

7      +solver/          % 求解算法
8      GradientSolver.m  % 直接法梯度求解器（整合全部方向优化/步长准则/投影/KKT）
9      LineSearch.m      % 非精确步长搜索（simple/Armijo/Wolfe）
10     +vis/             % 迭代过程可视化
11     Visualizer.m       % 实时四联图 + 动画录制（GIF/MP4）
12     +continuation/    % 同伦延拓外壳
13     HomotopyContinuation.m
14     demo/
15     demo_gradient.m    % 用梯度工具箱求解量子比特重置
16     demo_prolong.m     % 同伦延拓（prolong）求解
17     demo_example1.m    % 换一个可替换问题（例1）
18     demo_animation.m   % 边求解边动画录制成 GIF/MP4
19     tests/
20     validate_suite.m   % 自检（梯度下降方向、收敛、延拓）
21     descriptor/
22     main.tex           % 本文档

```

9.1 核心接口：ocp.AbstractOCP

一个具体问题实现 `costGradient`（目标与梯度）、`initialControl`（初始猜测）、`project/controlBounds`（可行域与投影）、`timeGrid`、`initialState` 等抽象方法即可。

9.2 核心求解器：solver.GradientSolver

求解器通过 `direction` 选择第5节的方向优化方法，通过 `lineSearch` 选择第6节的步长准则，通过 `kktActiveSet` 开启投影梯度的活动集处理，通过 `useProjection` 决定是否投影。它不关心问题物理，因而可用于任何 `AbstractOCP`。

9.3 延拓外壳：continuation.HomotopyContinuation

见第8节。它把问题沿一条参数路径延拓求解，用于克服梯度消失与初值敏感。

9.4 迭代过程可视化与动画：vis.Visualizer

求解器通过 `observer` 回调在每轮迭代对外报告当前的 `(iter,J,X,Mu,U,tg)`。`vis.Visualizer` 利用该回调实时刷新"状态、协态、控制、成本历史"四联图，并把每一帧录制成动画（可换的 gif 或 mp4）。这正是自研求解器相对于现成商用求解器的重要优势：能够输出并观察迭代的中间过程，从而定位收敛问题。示例见 `demo_animation.m`：

```
1 v = vis.Visualizer('animate', true, 'format', 'gif', 'videoFile', 'iteration.gif');
2 s.observer = @(iter,J,X,Mu,U,tgrid) v.snapshot(iter,J,X,Mu,U,tgrid);
3 res = s.solve(); v.finish();
```

9.5 如何替换"待求的最优控制问题"

只需新建一个继承 `ocp.AbstractOCP` 的类并实现抽象方法,然后 `solver.GradientSolver` 即可直接使用。`ocp.ControlExample` 给出了一个最小示例 (见 `demo_example1.m`)。

9.6 运行方式

将 `src` 加入 MATLAB 路径即可使用各包, 例如:

```
1 addpath('.../SigmaX_OCP/src');
2 model = ocp.QubitReset('tau', 0.15, 'N', 100);
3 s = solver.GradientSolver(model); s.direction = 'adam';
4 res = s.solve();
```

完整示例见 `demo/`, 自检脚本见 `tests/validate_suite.m`.